The Curious Coder

Part I - Python Foundations



Copyright

The Curious Coder — Part I: Python Foundations
First Edition

Copyright © 2026 Md Nasrullah. All rights reserved.

Terms of Use and Notice of Rights

This book is protected by copyright law. Unauthorized copying, sharing, or selling of this book, in whole or in part, is prohibited.

You may not, without the prior written permission of the author:

- Reproduce, copy, or scan this book, in print or digital form.
- Share, upload, or distribute it, including through file-sharing sites, messaging groups, cloud drives, and social media.
- Sell, resell, rent, lend for profit, or otherwise offer it for payment, alone or bundled with other products.
- Translate, adapt, or republish it, or present its text, structure, or exercises as your own work.
- Use it to train, fine-tune, or build datasets for machine-learning or AI systems.

Brief quotations in reviews and educational discussions are allowed, as long as the book and its author are credited.

If you bought or downloaded this book from anyone other than the author or an authorized seller, your copy is unauthorized. Please buy a legitimate copy. Purchases support the work that goes into the next edition.

Code Examples

You may use the code examples in your own programs and learning projects. You may not republish them in bulk, as part of a competing book, course, or tutorial.

Reporting Piracy

To report an unauthorized copy, or to request permission to use material from this book, contact the author at crafty_shadow@proton.me.

Disclaimer

The author has worked to make the information in this book accurate and complete. The book and its code are provided "as is," without warranty of any kind, express or implied. The author is not liable for any direct, indirect, incidental, or consequential damages arising from the use of the code, software, or ideas in this book.

Trademarks

Minecraft is a trademark of Mojang Synergies AB and Microsoft. Python is a trademark of the Python Software Foundation. All other product and company names are the property of their respective owners. This book is an independent work. It is not affiliated with, endorsed by, or sponsored by any of them.

Welcome to The Curious Coder

Here is a complete Python program:

```
player = "crafter_2505"
wins = 25

if wins >= 20:
    print(f"{player} qualifies for the tournament.")
```

Output:

```
crafter_2505 qualifies for the tournament.
```

Four lines. The program stores two facts, checks a rule, and responds. By the end of Chapter 1, you will understand every part of it. A few chapters later, you will write programs like this on your own.

The Curious Coder teaches Python to people who have never programmed. Every program starts as an idea. This book shows you how to turn yours into something that runs.

You will not memorize syntax lists. You will learn to break a problem into small steps, test each step, and fix what goes wrong. That skill outlasts any one language.

No experience is required. Curiosity and a little patience are enough.

Python Foundations

Python is a good first language. Its code reads close to plain English, and people use it for websites, automation, data analysis, and security tools. What you learn here carries over to all of them.

The book starts with what programming is and how to set up Python. From there, you work through variables, data types, conditions, loops, collections, functions, and files. Each chapter uses what the one before it taught, so nothing arrives out of nowhere.

Most examples follow Minecraft PvP players, ranks, and tournaments. They are small, familiar, and easy to picture, which keeps your attention on the code and off the setup.

When you finish, you will be able to write your own Python programs, work through a problem step by step, and keep learning without a textbook beside you.

How to Use This Book

You learn to program by writing code. Reading explains an idea, but typing it and changing it is what makes it stick.

Work through each chapter in five steps:

1. **Read.** Take in the idea and why it exists. Knowing the reason makes the rule easier to remember.
2. **Run.** Type the examples yourself and watch what happens. Typing is slower than copying, and that is the point.
3. **Experiment.** Change a value. Break a line. Guess the result first, then check. Most real understanding comes from this step.
4. **Solve.** Do the exercises without peeking at the answers. Struggling a little here is normal.
5. **Reflect.** Explain the idea in your own words. If you can't, reread the part that is fuzzy.

You do not need to get everything on the first pass. If something is hard, shrink the example and try again.

A Note About Mistakes

Errors are part of learning. They are not a sign that you are bad at this.

Every programmer hits bugs. Experienced ones just hit them with less panic. When your code fails, read the error message, because it usually names the problem. Then change one thing at a time and run it again.

This habit teaches you to investigate instead of guess. Over time you will spot mistakes faster and understand why they happened.

You are not aiming for perfect code on the first try. You are aiming to make progress when things go wrong.

Before You Begin

Install Python and pick a code editor. The setup chapter walks you through both if you have not done it yet.

Then do the work yourself. Type the examples, finish the exercises, and go back to earlier chapters whenever something slips. The learning is in the doing.

Stay curious while you do it. Ask "what happens if I change this?" and then try it.

Turn the page and write your first program.

About the Author

About the Author

Md Nasrullah

Developer and Cybersecurity Researcher
`Python • Offensive Security • OS Development`

Md Nasrullah is a developer and cybersecurity researcher. He works mainly in Python, offensive security, and low-level systems, and he wrote ***The Curious Coder — Part I: Python Foundations*** for people starting from zero.

He believes you learn programming by building things and breaking them. That belief shapes this book: every chapter asks you to run code, change it, and see what happens.

His own work covers:

- **Low-level systems:** system utilities and operating system development.
- **Cybersecurity:** network security testing, penetration testing, and vulnerability assessment.
- **Application engineering:** practical Python tools, scripts, and modular design.
- **Learning by teaching:** turning hard ideas into small examples a beginner can run.

What connects these areas is one question: how does this actually work? Nasrullah asks it all the way down, from the application to the hardware beneath it. This book is built to help you start asking it too.

The Curious Coder Philosophy

1. **Learn by building.** Understand an idea by using it in working software.
2. **Look underneath.** Syntax is only the surface. Ask what the machine is really doing.
3. **Experiment without fear.** A broken program is information, not failure.
4. **Stay curious.** Keep asking deeper questions, and keep building.

Find the Author

Questions, ideas, and updates on new editions are welcome.

- **YouTube (programming, security, and tech):** crafty_security
- **YouTube (Minecraft):** crafter_2505
- **Instagram:** crafter_2505
- **TikTok:** crafter_2505
- **Discord:** crafter_2505yt

Table of contents

1 What Is Programming? 7

2 Getting Started with Python 16

3 Your First Python Programs 23

4 Values, Variables, and Names 36

5 Data Types and Type Conversion 61

6 Input, Output, and User Interaction 91

7 Operators and Expressions 121

8 Working with Text and Strings 150

9 Making Decisions with Conditions 191

10 Boolean Logic and Comparisons 223

1 What Is Programming?

1.1 Welcome to the World of Programming

Imagine opening a video game, searching the internet, sending a message to a friend, or watching a video. These activities seem completely different, yet they share something: a digital device is doing work in response to instructions.

A game needs to know how its world behaves. A search engine finds useful information. A messaging application delivers messages to the right people. A video player shows images and plays sound at the right moment.

None of these systems understands its purpose the way a human does. They work because people have written instructions that tell computers how to perform specific tasks.

This is where programming comes in.

Programming is the process of creating instructions that a computer can carry out to perform tasks or solve problems.

A person who creates these instructions is called a **programmer**. Programmers build games, applications, websites, scientific tools, operating systems, robots, and much more.

Programming is not limited to complicated tasks. It can also automate repetitive work, organize information, perform calculations, and help people explore ideas.

Programming turns an idea into something a computer can actually do.

1.2 Computers Need Instructions

Take an ordinary situation: you want to make a cup of tea.

You already know what to do. Prepare water, heat it, add tea, wait for it to brew, and pour it into a cup.

For someone who knows the process, these steps seem obvious. Now imagine explaining the same task to a person who has never made tea.

Saying “make tea” might not be enough. They may not know which ingredients are needed, when to heat the water, or how long to wait. They need instructions.

Computers have a similar limitation, though the reasons differ. A computer does not automatically know what you intend when you ask it to accomplish a task. Its behavior depends on the instructions and systems it has been given. To make a computer perform a task, you must represent that task in a form the computer can process.

A music application, for example, needs instructions for selecting a song, starting playback, adjusting volume, and responding when a listener pauses the music.

A game needs rules for movement, collisions, scoring, and other events. Without instructions defining these behaviors, the game cannot operate as intended.

The principle is simple:

> Computers perform tasks according to instructions and rules; programmers design those instructions and rules to achieve a goal.

A computer's behavior also depends on its hardware, operating system, available resources, and other software. A program can fail if its instructions are incorrect or if the environment cannot support what it tries to do.

Understanding the relationship between instructions and behavior is one of the first steps toward understanding programming.

1.3 What Is a Program?

A **program** is a collection of instructions that a computer can execute to perform a task or a set of related tasks.

A digital game may contain many programs and components working together. Some handle movement, others manage sound, and others save progress or communicate with a server.

Each component has a purpose. Together, they create the experience the player sees.

Programs can be small or enormous. A small program might perform a single calculation. A large software system might contain millions of lines of instructions and involve many developers.

Size does not determine usefulness. A small tool that solves a problem reliably can be just as valuable as a much larger application.

Programs can also behave differently depending on the situation. A calculator application does not always perform the same operation. It responds to the numbers and operations its user selects.

This is possible because programs contain rules that determine which actions to take under different circumstances.

A program is therefore more than a random collection of instructions. It is an organized description of how a computer should behave to accomplish a particular goal.

1.4 Programming Is More Than Writing Instructions

It is easy to imagine programming as simply telling a computer what to do. Reliable software requires much more.

Before writing instructions, a programmer needs to understand the problem.

Suppose someone wants to build a school library management system. The goal is not merely to create a screen with buttons. The system must represent books, track which ones are available, record loans, and help librarians find information.

The programmer first identifies what the system needs to accomplish. Then they determine how to organize the information, which rules apply, and what happens when users perform different actions.

After that, the programmer designs a solution, expresses it in a programming language, tests whether it works, and improves it when problems appear.

This process involves several connected activities:

- **Understanding:** Identifying the problem and the result that is needed.
- **Planning:** Deciding how to approach the problem.
- **Designing:** Organizing the steps, information, and rules required.
- **Implementing:** Turning the planned solution into instructions a computer can execute.
- **Testing:** Checking whether the solution behaves as expected.
- **Improving:** Correcting mistakes and making the solution more reliable or useful.

These activities do not always happen in a straight line. Programmers often return to earlier stages after discovering a problem or learning something new.

Testing, for example, may reveal that a requirement was misunderstood. The design may then need to change before implementation can continue.

Programming is both a technical activity and a problem-solving process.

1.5 Programming Languages: How Humans Communicate Instructions to Computers

People communicate using human languages such as English and Bangla. These languages are flexible and can express complex ideas, but they can also be ambiguous.

A computer needs instructions expressed in forms that its processing systems can interpret according to precise rules.

A **programming language** provides a structured way to express those instructions.

Python, C, C++, Java, and JavaScript are examples. Each has its own rules, features, and common uses.

Some languages are particularly useful for systems programming. Others are widely used for web development, data analysis, or automation. Many can be used in several areas.

Every programming language has rules governing how instructions are written and what they mean. These rules determine whether a program is valid and how it behaves.

Learning a programming language is a little like learning a new way to express ideas. You learn its vocabulary, its rules, and how to combine its elements to communicate a solution.

Still, programming languages are not ordinary human languages with different words. They are designed to express computational operations precisely.

1.6 Why Are There Different Programming Languages?

No single programming language is ideal for every situation.

Some languages offer greater control over computer memory and hardware. Others emphasize readability or provide convenient tools for particular kinds of work. Languages also differ in their libraries, performance characteristics, and development environments.

Python is known for its readable syntax and broad range of applications. C is often used when low-level control and efficient use of system resources matter. JavaScript is widely used to build interactive web pages.

These are common uses, not strict boundaries. Python can be used for systems-related tools, C can build application components, and JavaScript can run outside the browser.

In this book, you will learn Python as your first practical programming language. The deeper goal is to learn how to solve problems in a way that transfers to other languages.

1.7 How Does a Computer Understand a Program?

Computers perform operations using electronic hardware. At the most fundamental level, digital systems represent information as binary values, commonly written as zeros and ones.

Programmers usually do not write ordinary applications directly as long sequences of binary values. They use programming languages and tools that transform their instructions into forms suitable for execution.

Different languages take different approaches.

Some languages are **compiled** into machine code before execution. Others are commonly **interpreted** or run in a runtime environment. Some systems combine the two.

The most common Python implementation translates source code into an intermediate form called **bytecode**, which the Python virtual machine then executes. Other Python implementations may work differently.

A programming language provides a way to express instructions, and software tools and runtime systems make those instructions executable on a computer.

You do not need to understand every detail of computer hardware before writing your first program. Those details matter more as you study memory, processors, operating systems, and performance.

For now, remember that your instructions pass through a software and hardware system before producing the behavior you expect.

1.8 What Is an Algorithm?

To create a program, you first need a way to describe how a problem should be solved.

An **algorithm** is a defined sequence of steps or rules for solving a problem or completing a task.

Algorithms are not limited to computers. People use them in everyday life whenever they follow a procedure to achieve a result.

Take the problem of finding the largest number in a collection. A simple approach is to examine the numbers one at a time while keeping track of the largest value seen so far. When every number has been examined, the stored value is the largest.

This method solves the problem without committing to any particular programming language.

An algorithm can be described in ordinary language, diagrams, pseudocode, or other forms. Later, it can be translated into a programming language so that a computer can carry it out.

1.9 Algorithms and Programs Are Not the Same

The two terms are closely related, but they mean different things.

An **algorithm** describes a method for solving a problem. A **program** contains instructions a computer can execute, often implementing one or more algorithms.

A single algorithm can be implemented in many programming languages. The method stays the same even when the instructions look different.

A program may also contain many algorithms alongside other components, such as data storage, input handling, and communication with external systems.

Learning to separate the problem-solving method from its implementation will deepen your understanding of programming.

1.10 What Does It Mean to Think Like a Programmer?

Programmers need more than knowledge of a language's rules. They also need a particular approach to solving problems.

This approach is called **computational thinking**.

It involves breaking problems into manageable parts, finding useful patterns, focusing on relevant information, and developing clear procedures for reaching a solution.

It does not mean thinking like a computer or giving up creativity. It means approaching problems in a structured way that makes solutions easier to understand, implement, and test.

Four ideas support this way of thinking.

1.10.1 Decomposition: Breaking a Problem into Smaller Parts

Imagine building an online store. Designing the entire system at once would be overwhelming.

Instead, divide it into smaller components: displaying products, managing a shopping cart, processing orders, tracking inventory, handling customer accounts, and sending order confirmations.

Each component is a smaller problem. Once you understand those problems, you can combine their solutions into a larger system.

This approach is called **decomposition**.

It makes complex tasks easier to reason about. It also lets programmers work on one part of a system without constantly considering every other part.

1.10.2 Pattern Recognition: Finding Similarities

Suppose you need to calculate the total price of several products. The same general idea applies to each product: find its price and account for how many items are being purchased.

Recognizing repeated structures helps you build a general solution instead of solving each case separately.

Pattern recognition lets programmers reuse ideas, avoid unnecessary repetition, and see relationships between different problems.

1.10.3 Abstraction: Focusing on What Matters

Imagine using a digital map. You see roads, locations, and directions without needing to understand the electronic circuitry or the internal calculations behind the application.

The map hides many implementation details and presents only the information relevant to your task.

This is **abstraction**: focusing on the essential features of a problem while temporarily ignoring details that are not currently important.

Abstraction helps programmers manage complexity. They can work with a component based on what it does, without understanding every internal detail at once.

1.10.4 Step-by-Step Reasoning: Making the Solution Clear

A solution should be clear enough that its steps can be examined and checked.

If a task involves several decisions, the programmer must determine what happens in each situation and how each decision affects the result.

Working through these steps uncovers missing cases, incorrect assumptions, and confusing logic before they turn into difficult software problems.

Together, these techniques help programmers turn vague ideas into solutions that can be implemented and evaluated.

1.11 Programming Requires Creativity and Logic

Programming is often associated with mathematics and logical reasoning. Those skills help, but programming is not limited to mathematical problems.

A programmer might create a game mechanic, design a useful application, automate a tedious task, or build a tool that helps people organize their work.

These activities require **creativity** because there are usually many possible solutions. The programmer decides which approach best fits the goal, the users, and the available resources.

Logic matters just as much. A solution that sounds exciting still has to work correctly. Its rules must be consistent, its instructions must be meaningful, and its behavior must be tested.

Good programming combines imagination with careful reasoning.

You do not need to be an expert mathematician or know everything about computers before you begin. You need curiosity, patience, and a willingness to break problems into smaller, understandable steps.

As your experience grows, you will develop the technical skills to turn more ambitious ideas into working software.

1.12 Bugs, Errors, and Improvement

Programs do not always behave as their creators expect.

A game might calculate a score incorrectly. A website might display the wrong information. An application might stop working when it meets an unexpected situation.

A mistake in a program is called a **bug**. Locating and correcting problems is called **debugging**.

Bugs arise for many reasons. The programmer might misunderstand a requirement, design an incorrect algorithm, misuse a language feature, or overlook an unusual situation.

Some problems are easy to identify. Others appear only under particular conditions and require careful investigation.

Professional programmers do not simply hope their programs are correct. They rely on testing, debugging tools, code reviews, and other techniques to build confidence in their software.

Even when a program appears to work, it may contain problems that have not yet been discovered.

For this reason, programming involves continuous improvement. A solution can be revised to handle more situations, become easier to understand, run more efficiently, or serve its users better.

Mistakes are a normal part of learning programming. What matters is learning to investigate them, understand their causes, and improve your approach.

1.13 What Can You Build with Programming?

Programming is used in an enormous range of fields. The same fundamental ideas support very different kinds of projects.

- **Games:** Programmers create game rules, movement systems, artificial intelligence, graphics, multiplayer features, and tools used by other game developers.
- **Websites and web applications:** Programming builds interactive websites, online stores, communication platforms, dashboards, and services that run over the internet.
- **Desktop applications:** Programs help people edit images, manage files, organize projects, and perform specialized tasks.
- **Automation:** Software can handle repetitive activities, reducing manual work and making processes more consistent.
- **Science and engineering:** Programs help researchers analyze data, simulate physical systems, control equipment, and investigate complex questions.
- **Operating systems and system software:** Programmers build the software that manages computer resources, supports applications, and interacts with hardware.
- **Artificial intelligence:** Programming is used to develop systems that recognize patterns, process language, make predictions, and perform other tasks suited to computational models.

These are only a few examples. Programming is a tool for solving problems, so its applications depend on the problems people want to address.

You do not need to choose a specialization now. Exploring different areas as you learn will show you which kinds of projects interest you most.

1.14 Your Journey as a Programmer

You now understand what programming is at a conceptual level. You have seen why computers need instructions, what programs and algorithms are, how programming languages fit into the process, and why problem-solving is central to software development.

Programming is not about memorizing commands. It involves understanding problems, planning solutions, expressing ideas precisely, testing results, and improving what you build.

In the chapters ahead, you will turn these ideas into practical skills. You will begin learning Python and see how programs receive and process information, make decisions, repeat operations, and organize data. Eventually you will build complete projects.

You do not need to understand everything at once. Each concept provides a foundation for the next.

The goal is not to learn just enough Python to copy examples. It is to develop the ability to reason about problems and create your own solutions.

That is what makes you a programmer.

1.15 Chapter 1 Review

Before moving on, make sure you can explain the following concepts in your own words:

- What programming is and why computers need instructions.
- What a program is and how it differs from an algorithm.
- Why programming languages provide structured ways to express instructions.
- How algorithms help describe solutions before implementation.
- What computational thinking means.
- How decomposition, pattern recognition, and abstraction help solve problems.
- Why creativity, logic, testing, and debugging are important in programming.

You do not need to memorize every sentence in this chapter. Focus on understanding the ideas well enough to explain them to someone else.

3 Your First Python Programs

You have seen what programming is, and your computer is ready for Python. Time to write some code.

A program does not need to be complicated to be useful. One instruction can display a message, perform a calculation, or save someone a few minutes of work.

This chapter covers writing simple Python instructions, displaying information, adding comments, and following the basic rules Python expects. By the end, you can create, run, and troubleshoot small programs of your own.

3.1 Your First Line of Python

Create a file named `hello.py` in your code editor and write this:

```
print("Hello, world!")
```

Save the file and run it with Python. You should see:

```
Hello, world!
```

Congratulations. You have written your first Python program.

It is tiny, but it shows something big: a computer follows instructions written in a language it can execute.

Here is what each part does.

- `print` is a built-in Python function that displays information.
- The parentheses `()` hold the information you pass to the function.
- The quotation marks tell Python that `Hello, world!` is text.

Text inside quotation marks is called a **string**. A later chapter covers strings in depth.

When Python runs this instruction, the text appears in the output area of your programming environment.

3.2 Understanding print()

`print()` is one of a beginner's most useful tools. It displays text, numbers, and the results of expressions.

3.2.1 Displaying Text

To display text, put it inside quotation marks within `print()`.

```
print("Welcome to Python!")
print("I am learning to program.")
print("This is my first project.")
```

Output:

```
Welcome to Python!
I am learning to program.
This is my first project.
```

Each call to `print()` normally starts a new line of output.

The quotation marks do not appear in the output. They only mark where the text begins and ends.

Python accepts both double and single quotation marks:

```
print("Hello!")
print('Welcome!')  # Single quotes also work
```

Both instructions are valid. This book uses double quotation marks, and you should pick one style and stick with it. A later chapter explains what happens when your text contains apostrophes or other special characters.

3.2.2 Displaying Numbers

`print()` also displays numbers. Leave off the quotation marks.

```
print(25)
print(3.14)
print(-8)
```

Output:

```
25
3.14
-8
```

Python treats a number without quotation marks as a numeric value, not text. The difference matters once you start calculating.

```
print(10 + 5)
print(20 - 7)
print(6 * 4)
```

Output:

```
15
13
24
```

Python works out each expression first, then displays the result.

You do not need to memorize operators yet. Just remember that `print()` can show the result of an operation, not only a message.

3.2.3 Displaying Multiple Values

Pass several values to one `print()` call by separating them with commas.

```
print("Age:", 14)
print("Score:", 95)
print("Total:", 10 + 20)
```

Output:

```
Age: 14
Score: 95
Total: 30
```

Python displays the values in order and puts a space between them.

This is handy for pairing a label with a number or a calculation result. A simple game could show a player's stats this way:

```
print("Player:", "crafter_2505")
print("Score:", 150)
print("Bonus:", 25)
```

Output:

```
Player: crafter_2505
Score: 150
Bonus: 25
```

Three short instructions, and the output already looks like a scoreboard.

3.3 Python Follows Instructions in Order

By default, Python runs a program from top to bottom.

```
print("Starting the program")
print("Loading information")
print("Program finished")
```

Output:

```
Starting the program
Loading information
Program finished
```

The first instruction runs first, the second runs second, and so on.

Python never rearranges instructions because of what their messages say. It simply follows the order you wrote them in.

Swap the lines and watch what happens:

```
print("Program finished")
print("Starting the program")
print("Loading information")
```

Output:

```
Program finished
Starting the program
Loading information
```

The program now announces that it has finished before it has started. Same instructions, different order, different result.

Order is fundamental to programming. Later, conditions, loops, and function calls give you control over which instructions run and when.

3.4 Giving Your Program a Personality

A program can display anything you like, as long as the instructions are valid. A short introduction makes a good example:

```
print("========================")
print("   MY CODING JOURNEY")
print("========================")
print("Name: Nasrullah")
print("Goal: Learn Python")
print("Current level: Beginner")
print("========================")
```

Output:

```
========================
   MY CODING JOURNEY
========================
Name: Nasrullah
Goal: Learn Python
Current level: Beginner
========================
```

The lines of equals signs are ordinary text. Because they sit inside quotation marks, Python does not treat them as formatting commands.

Basic tools can present information clearly and attractively. You could build a game introduction, a digital greeting card, a scoreboard, or a menu for a future project.

Programming is not only calculation. It is also a way to express ideas.

3.5 Comments: Notes for Humans

As programs grow, you want to explain what each part is for. Python supports comments for this.

A comment starts with the # character. Python ignores everything after it on that line.

```
# Display a welcome message
print("Welcome to The Curious Coder!")

# Display the current learning goal
print("Learning Python step by step")
```

Output:

```
Welcome to The Curious Coder!
Learning Python step by step
```

The comments never appear in the output. They exist for the people reading the code.

3.5.1 Why Comments Matter

Picture yourself opening a program you wrote months ago. You may remember what it does, but not why you wrote a particular line. A good comment answers that question.

```
# Show the player their final score
print("Final score:", 250)
```

This comment gives the purpose of the instruction. It does not just repeat the code.

Comments help in several situations:

- Explaining why you chose a particular approach.
- Describing a complicated section of code.
- Leaving a reminder about something to improve.
- Helping another programmer understand your work, whether that is a teammate or your future self.

Use them with care. A comment that repeats the obvious only adds clutter.

```
# Print hello
print("Hello")
```

The instruction already says it all. A better comment gives the reason behind the message:

```
# Welcome new players when the application starts
print("Welcome to the game!")
```

3.5.2 Comments Can Appear Beside Instructions

A comment can also follow an instruction on the same line.

```
print("Welcome!")  # Display the welcome message
```

Python runs the instruction and ignores everything after the #. The exception is a # inside a string:

```
print("The # symbol starts a comment")
```

Output:

```
The # symbol starts a comment
```

Here the # sits inside quotation marks, so it is ordinary text.

3.6 Understanding Basic Python Syntax

Every programming language has rules for how instructions must be written. These rules are called **syntax**.

Break them and Python cannot understand your program. Four common cases show how this works.

3.6.1 Parentheses Matter

This is valid Python:

```
print("Hello!")
```

This is not:

```
print("Hello!"  # BUG: missing closing parenthesis
```

The second example never closes its parenthesis, so Python reports a syntax error instead of running it.

When you use paired symbols such as parentheses or quotation marks, close every one you open.

3.6.2 Quotation Marks Matter

This is valid:

```
print("Learning Python")
```

This is not:

```
print("Learning Python)  # BUG: missing closing quotation mark
```

The opening quotation mark has no partner. Python cannot tell where the string ends, so it reports an error.

3.6.3 Spelling and Capitalization Matter

Python knows the function as `print`, not `Print` or `PRINT`.

```
print("This works")
```

These do not work:

```
Print("This fails")  # BUG: wrong capitalization
PRINT("This fails")  # BUG: wrong capitalization
```

Python is **case-sensitive**. It treats uppercase and lowercase letters as different characters. The same rule applies to names you create later, including variables and functions.

This kind of mistake produces a `NameError`, not a syntax error. Python understands the structure of the line but does not recognize the name.

3.6.4 Extra Punctuation Is Unnecessary

Python does not need a semicolon at the end of an ordinary instruction.

```
print("Hello!")
print("Goodbye!")
```

This is the standard style. Python does allow semicolons in some situations, but they make beginner code harder to read. Follow conventional Python formatting and keep each instruction on its own line.

3.7 Reading Error Messages

Everyone makes mistakes. Experienced developers write code that fails on the first try all the time.

The skill that matters is investigating the problem. Suppose you run this program:

```
print("Hello, world!"  # BUG: missing closing parenthesis
```

Python responds with an error message like this:

```
  File "hello.py", line 1
    print("Hello, world!"
          ^
SyntaxError: '(' was never closed
```

The exact wording depends on the error and on your Python version, but most messages contain the same parts:

- **Error type:** The category of problem Python detected, such as `SyntaxError`.
- **File name:** The file where the problem occurred.
- **Line number:** The approximate location of the problem.
- **Description:** A short explanation of what went wrong.

3.7.1 A Simple Debugging Process

When a program fails, work through these steps:

1. Read the error message carefully.
2. Find the line it mentions.
3. Check punctuation, quotation marks, spelling, and structure.
4. Make one correction at a time.
5. Run the program again and see whether the error is gone.

The line in the message is not always the true source of the mistake. Sometimes Python only notices the problem when it reaches a later instruction. An unclosed quotation mark on one line, for example, can trigger an error on the line after it.

Learning to trace these problems is part of becoming a capable programmer.

3.8 Syntax Errors and Logic Errors

Not every mistake is the same kind of mistake.

A **syntax error** means the code breaks the language's rules, so Python cannot run it:

```
print("Hello"  # BUG: missing closing parenthesis
```

A **logic error** means the program runs but gives a result you did not intend:

```
# Goal: add 10 and 3
print(10 - 3)  # BUG: subtracts instead of adding
```

Output:

```
7
```

This code is valid, and Python runs it without complaint. But the goal was addition, so the correct answer is 13. The program is wrong even though nothing crashed.

That is the key difference:

- Syntax errors stop Python from running the affected code.
- Logic errors let the program run while quietly producing the wrong result.

Later chapters cover debugging and the different kinds of errors in much more depth.

3.9 A First Look at Python's Interactive Mode

Python can also run instructions one at a time, interactively. This environment is called the Python interpreter or the **REPL**, short for Read-Eval-Print Loop.

Instead of saving a whole program in a file, you type an instruction and see the result at once.

Start Python in interactive mode. You see a prompt that looks like this:

```
>>>
```

Type an instruction after the prompt and press Enter. Python responds immediately:

```
>>> print("Hello from the interpreter!")
Hello from the interpreter!
>>> print(4 + 6)
10
>>> print("Experimenting is useful")
Experimenting is useful
```

Each instruction runs as soon as you enter it, and the session stays open for the next one.

Interactive mode is useful for:

- Testing small pieces of code.
- Checking how an expression behaves.
- Exploring unfamiliar Python features.
- Experimenting without creating a new file every time.

It does not replace `.py` files, though. Files let you save your work, organize larger programs, and run the same instructions again later. You will use both approaches as you continue.

3.10 Building Your First Complete Program

Now combine what you have learned. Create a file called `my_introduction.py` and enter this code:

```
# My first complete Python program

print("==============================")
print("       ABOUT ME")
print("==============================")

print("Name: Nasrullah")
print("Favorite subject: Computer Science")
print("Current goal: Learn Python")
print("Future goal: Build useful software")

print("==============================")
print("Thanks for reading!")
```

Run it and look at the output. Several instructions work together to produce a readable layout, and a comment records what the program is for.

Then make it yours. Change the name, subject, goals, and closing message. Add lines. Remove some of the decoration. Try a different layout.

You do not need advanced Python to create something that belongs to you. Small projects are one of the best ways to practice what you already know.

3.11 Good Habits from the Beginning

Even simple programs benefit from good habits.

3.11.1 Use Meaningful File Names

Choose names that describe what a program does:

```
hello.py
my_introduction.py
game_menu.py
scoreboard.py
```

Avoid vague names like `thing.py` or `stuff.py` when a descriptive one is available. You will thank yourself when you have thirty files.

3.11.2 Keep Your Code Readable

Arrange instructions so a person can follow them easily. Compare these two programs:

```
print("Welcome")
print("Choose a game")
print("Good luck")
```

```
print("Welcome");print("Choose a game");print("Good luck")  # Valid, but hard to read
```

Both are valid Python. The first is far easier to read and change.

Readable code matters more as your programs grow.

3.11.3 Save Your Work

Save programs as `.py` files so you can return to them later. A dedicated folder for your practice projects keeps things tidy:

```
python-projects/
    hello.py
    my_introduction.py
    game_menu.py
```

A consistent folder structure prevents confusion as the number of programs grows.

3.11.4 Experiment Safely

When you try something unfamiliar, change one small thing and watch what happens.

If the program breaks, undo your last change or read the error message. Keeping a working copy before a big change is a cheap safety net.

Experiments teach the most when you pay attention to what changed and why.

3.12 Chapter Review

You have written and run your first Python programs. This chapter covered:

- The `print()` function and how it displays information.
- Printing text, numbers, and calculation results.
- Passing multiple values to `print()`.
- The order in which Python runs instructions.
- Comments and their purpose.
- Basic Python syntax and common mistakes.
- Reading error messages and telling syntax errors from logic errors.
- Python's interactive mode and the REPL.
- Creating, saving, and organizing simple `.py` files.
- Good programming habits.

These ideas are the foundation for everything that follows. Soon you will combine them with variables, conditions, loops, functions, and data structures to build more powerful programs.

3.13 Practice and Challenges

Complete these exercises on your own. Use only the concepts you have learned so far.

3.13.1 Challenge 1: The Welcome Message

Create a program named `welcome.py` that displays:

- A welcome message.
- The name of the book you are studying.
- The programming language you are learning.
- A short sentence describing your goal.

Use a separate `print()` call for each line.

3.13.2 Challenge 2: The Information Card

Create a program that displays a neatly formatted information card.

Include at least five pieces of information, such as a name, a favorite subject, a hobby, a programming goal, and a favorite technology. Use decorative characters to separate the sections.

3.13.3 Challenge 3: The Calculator Display

Write a program that displays the results of at least five arithmetic expressions.

Use addition, subtraction, and multiplication, and add descriptive labels so the output is easy to understand.

3.13.4 Challenge 4: The Code Detective

Create a Python file that contains several deliberate mistakes: a missing parenthesis, an incorrectly capitalized `print`, an unclosed quotation mark.

Run the program and study the error messages. Then fix the mistakes one at a time until it runs. Write down what each error taught you.

3.13.5 Challenge 5: Improve a Program

Take the complete introduction program from this chapter and customize it.

Add at least three new lines of information, improve the layout, and add useful comments. The result should be readable and personal.

3.13.6 Challenge 6: Predict the Output

Without running the following program, write down what you think it displays:

```
print("First")
print(2 + 3)
print("Third")
print(10 - 4)
```

Then run it and compare. If your prediction was wrong, find the instruction you misunderstood.

3.13.7 Challenge 7: Design Your Own Program

Create a small program that communicates an idea of your choice. It could be a game welcome screen, a fictional shop receipt, a simple scoreboard, a motivational message, or an introduction to a project you want to build.

Use at least six `print()` calls and one comment, and make the output well organized.

3.13.8 Challenge 8: Explain Your Code

Choose one of your programs and explain, in your own words:

- What the program does.
- What each instruction contributes.
- Why the instructions appear in their current order.
- What would happen if you changed one instruction.
- How you would investigate an error if the program stopped working.

Understanding your code matters as much as getting it to run.

Coming up next: Chapter 4 covers values, variables, and names. You learn how programs store information and refer to it, so they can work with data instead of displaying only fixed messages.

Enjoyed the sample?

This was a free sample of The Curious Coder — Part I: Python Foundations.
It contains Chapter 1 and Chapter 3. The full book has 10 chapters,
with practice questions and answers at the end of each one.

In the full book

1 What Is Programming?
2 Getting Started with Python
3 Your First Python Programs
4 Values, Variables, and Names
5 Data Types and Type Conversion
6 Input, Output, and User Interaction
7 Operators and Expressions
8 Working with Text and Strings
9 Making Decisions with Conditions
10 Boolean Logic and Comparisons

Questions or permission requests: crafty_shadow@proton.me

© 2026 Md Nasrullah. This sample may be shared freely in its complete, unchanged form.